

CSE 4403: Algorithms

Lecture14

Dynamic Programming – III

Part-1

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: anikafarzana@iut-dhaka.edu

Two Necessary Ingredients

INGREDIENT 1

Overlapping Subproblems

The same smaller problem appears many times. Without this, DP gives no benefit over plain recursion.

INGREDIENT 2

Optimal Substructure

The best solution to a big problem is built from best solutions of smaller problems.

The DP Recipe

How to solve a DP problem

- Define the state
- Define the recurrence
 - Means: How to combine the previous results to get current?
- Identify base cases
- Compute/store answers
- Return the desired state

Number of Permutations

Problem:

- You have coins: 1, 3, 4
- Find the number of ways to make 6.
- **A coin denomination may be used any number of times.**

- **State**

$dp[x]$ = number of ways to make amount x

- **Base Case**

There's exactly one way to make amount 0:

$dp[0] = 1$

Number of Permutations

Problem reformulated:

- Given coins of different denominations and a target amount, count the number of **distinct permutations** that sum to the target. **For now, lets say order matters — {1,2} and {2,1} are not the same.**

- **State**

$dp[x]$ = number of ways to make amount x

- **Base Case**

There's exactly one way to make amount 0:

$dp[0] = 1$

Number of Permutations

- **Recurrence**

For each coin:

$dp[x] += dp[x - \text{coin}]$

```
int main() {  
    int n = 6;  
    vector<long long> dp(n + 1, 0);  
  
    dp[0] = 1;  
  
    for (int i = 1; i <= n; i++) {  
  
        long long ways = 0;  
  
        if (i - 1 >= 0) ways += dp[i - 1];  
        if (i - 3 >= 0) ways += dp[i - 3];  
        if (i - 4 >= 0) ways += dp[i - 4];  
  
        dp[i] = ways;  
    }  
  
    cout << dp[n] << '\n';  
}
```

Number of Permutations

- **Recurrence**

For each coin:

$$dp[x] += dp[x - \text{coin}]$$

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    vector<int> coins = {1, 3, 4};
    int n = 6;

    vector<int> dp(n + 1, 0);
    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        for (int c : coins) {
            if (i >= c)
                dp[i] += dp[i - c];
        }
    }

    cout << dp[n] << '\n';
}
```

Coin Change: Number of Combinations (Order of coins doesn't matter)

- **Recurrence**

For each coin:

$dp[x] += dp[x - \text{coin}]$

```
int main() {  
    int target = 6;  
  
    vector<long long> dp(target + 1, 0);  
    dp[0] = 1;  
  
    // coin = 1  
    for (int i = 1; i <= target; i++)  
        dp[i] += dp[i - 1];  
  
    // coin = 3  
    for (int i = 3; i <= target; i++)  
        dp[i] += dp[i - 3];  
  
    // coin = 4  
    for (int i = 4; i <= target; i++)  
        dp[i] += dp[i - 4];  
  
    cout << dp[target] << '\n';  
}
```

Coin Change: Number of Combinations (Order of coins doesn't matter)

In other words:

The order of the loop has to be reversed.

```
int main() {  
    vector<int> coins = {1, 3, 4};  
    int n = 6;  
  
    vector<long long> dp(n + 1, 0);  
    dp[0] = 1;  
  
    for (int coin : coins) {  
        for (int sum = coin; sum <= n; sum++) {  
            dp[sum] += dp[sum - coin];  
        }  
    }  
  
    cout << dp[n] << '\n';  
}
```

Coin Change: Number of Combinations (Order of coins doesn't matter)

- **Why does coin outer, amount inner give combinations?**

When you process one coin fully before moving to the next, each combination is built in exactly one canonical order — smallest coin first. So {1,3} is only ever built as "ways to make 3 using coin=1, then add coin=3." The reverse order {3,1} is never separately constructed.

```
int main() {  
    vector<int> coins = {1, 3, 4};  
    int n = 6;  
  
    vector<long long> dp(n + 1, 0);  
    dp[0] = 1;  
  
    for (int coin : coins) {  
        for (int sum = coin; sum <= n; sum++) {  
            dp[sum] += dp[sum - coin];  
        }  
    }  
  
    cout << dp[n] << '\n';  
}
```

Coin Change/Number of Combinations (Order of coins doesn't matter)

Trace with coins=[1,3], target=4:

Start: dp = [1, 0, 0, 0, 0]

coin = 1:

sum=1: dp[1] += dp[0] → dp[1] = 1 (one 1)

sum=2: dp[2] += dp[1] → dp[2] = 1 (two 1s)

sum=3: dp[3] += dp[2] → dp[3] = 1 (three 1s)

sum=4: dp[4] += dp[3] → dp[4] = 1 (four 1s)

coin = 3:

sum=3: dp[3] += dp[0] → dp[3] = 2 (three 1s) OR (one 3)

sum=4: dp[4] += dp[1] → dp[4] = 2 (four 1s) OR (one 1 + one 3)

Answer: dp[4] = 2 → {1,1,1,1} and {1,3}

{3,1} was never separately counted. Contrast with permutations (amount outer), where dp[4] would also count {3,1} as a distinct sequence.

Coin Change: 2D version

We could have done this in 2D too but what's the use?.

```
vector<vector<long long>> dp(m + 1, vector<long long>(n + 1, 0));

// Base Case
dp[0][0] = 1;           // One way to make sum 0 with 0 coins

// Main DP
for (int i = 1; i <= m; i++) {
    for (int sum = 0; sum <= n; sum++) {

        // Don't take current coin
        dp[i][sum] = dp[i - 1][sum];

        // Take current coin (can reuse it)
        if (sum >= coins[i - 1])
            dp[i][sum] += dp[i][sum - coins[i - 1]];
    }
}

cout << dp[m][n] << '\n';
```

Coin change: Number of combinations

Coins = {1, 3, 4}

Target Sum = 5

You have to find the number of combinations.

i / Sum	0	1	2	3	4	5
0 coins	1	0	0	0	0	0
1 (coin = 1)	1	1	1	1	1	1
2 (coins = 1,3)	1	1	1	2	2	2
3 (coins = 1,3,4)	1	1	1	2	3	3

0/1 Knapsack

Problem: There are n items, each with **weight[i]** and **value[i]**. Knapsack capacity **W**.

You have to **maximize the value without exceeding capacity**. Each item should **be used at most once**.



0/1 Knapsack

Problem: There are n items, each with **weight[i]** and **value[i]**. Knapsack capacity **W**.

You have to **maximize the value without exceeding capacity**. Each item should **be used at most once**.

State: $dp[i][w]$ = max value using first i items with capacity w



0/1 Knapsack

Recurrence relation:

```
if weight[i] > w:  
    dp[i][w] = dp[i-1][w]        // can't fit, skip  
else:  
    dp[i][w] = max(dp[i-1][w],    // skip item i  
                   dp[i-1][w - weight[i]] + value[i]) // take item i
```

0/1 Knapsack

```
// Base Cases
for (int i = 0; i <= n; i++)
    dp[i][0] = 0;

for (int w = 0; w <= W; w++)
    dp[0][w] = 0;

// Main DP Loop
for (int i = 1; i <= n; i++) {
    for (int w = 1; w <= W; w++) {

        if (weight[i] > w)
            dp[i][w] = dp[i - 1][w];
        else
            dp[i][w] = max(dp[i - 1][w],
                           dp[i - 1][w - weight[i]] + value[i]);
    }
}

cout << dp[n][W] << endl;
```

0/1 Knapsack

```
int weight[] = {1, 3, 4};
```

```
int value[] = {2, 4, 5};
```

```
int n = 3;
```

```
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
1(w=1)	0	2	2	2	2	2
2(w=3)	0	2	2	4	6	6
3(w=4)	0	2	2	4	5	7

0/1 Knapsack

```
int weight[] = {1, 3, 4};
```

```
int value[] = {2, 4, 5};
```

```
int n = 3;
```

```
int W = 5;
```

item \ weight	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	2	2	2	2	2
2	0	2	2	4	6	6
3	0	2	2	4	5	7

0/1 Knapsack

Why did we need 2D DP here?

- In **0/1 Knapsack**, each item can be chosen **only once**.
- If we use the **current row**, then $dp[i][w - \text{weight}[i]]$ may already include the current item. Adding it again would count the same item **twice**, making it behave like **Unbounded Knapsack**.
- Therefore, we use the **previous row ($dp[i-1]$)** when including the current item, ensuring each item is used **at most once**.

0/1 Knapsack

Why did we need 2D DP here?

Example:

- While filling $dp[2][5]$ (considering the 2nd item) with $w[2] = 2$:
$$dp[2][5] = \max(dp[2][5], dp[2][5-w[2]] + val[2])$$
$$= \max(dp[2][5], dp[2][3] + val[2])$$
- But $dp[2][3]$ may **already include the 2nd item**, since it was computed using the **current row**.
Adding $val[2]$ again would **choose the same item twice**, which violates the **0/1** constraint.
- **Therefore, we use the previous row ($dp[i-1]$) instead of the current row.**

0/1 Knapsack: Optimization

Can we optimize it, can we skip 2D DP here?

One optimization could be using two rows only -> current row and previous row. But still its 2D.

```
vector<vector<int>> dp(2, vector<int>(W + 1, 0));

for (int i = 1; i <= n; i++) {
    for (int w = 1; w <= W; w++) {

        if (weight[i] > w)
            dp[i % 2][w] = dp[(i - 1) % 2][w];
        else
            dp[i % 2][w] = max(dp[(i - 1) % 2][w],
                               dp[(i - 1) % 2][w - weight[i]] + value[i]);
    }
}

cout << dp[n % 2][W];
```

0/1 Knapsack: 1D Backward Hack

But, the question is could we skip 2D DP here?

Yes by running the weight loop backward so that the previous row values are there when updating.

```
vector<int> dp(W + 1, 0);

for (int i = 1; i <= n; i++) {
    for (int w = W; w >= weight[i]; w--) {

        dp[w] = max(dp[w],
                    dp[w - weight[i]] + value[i]);
    }
}

cout << dp[W];
```

0/1 Knapsack: Reconstruction

When is 2D DP still useful?

When we want to know which items are picked for weight W knapsack.

We will see in Next Class!